

Gayatri

DBMS

①

~~DBMS~~

~~DBMS~~

Transaction Management

Sruvidya

Collections of operations that form a single logical unit of work is called a transaction.

A DB system must ensure proper execution of transactions despite failure. That is, either the entire transaction executes or none of it occurs.

Example: A transfer of fund from one account to another is a single operation from customer's point of view. But internally there are multiple operations involved - debit & credit.

Desirable properties of a transaction.

- ACID properties

Atomicity

Consistency

Isolation

Durability

Atomicity: Atomicity property ensures that any transaction that has begun executes till its completion or in case the transaction fails due to some reason, the database reverts the transaction (failed) to the state it was in before this failed transaction.

(2)

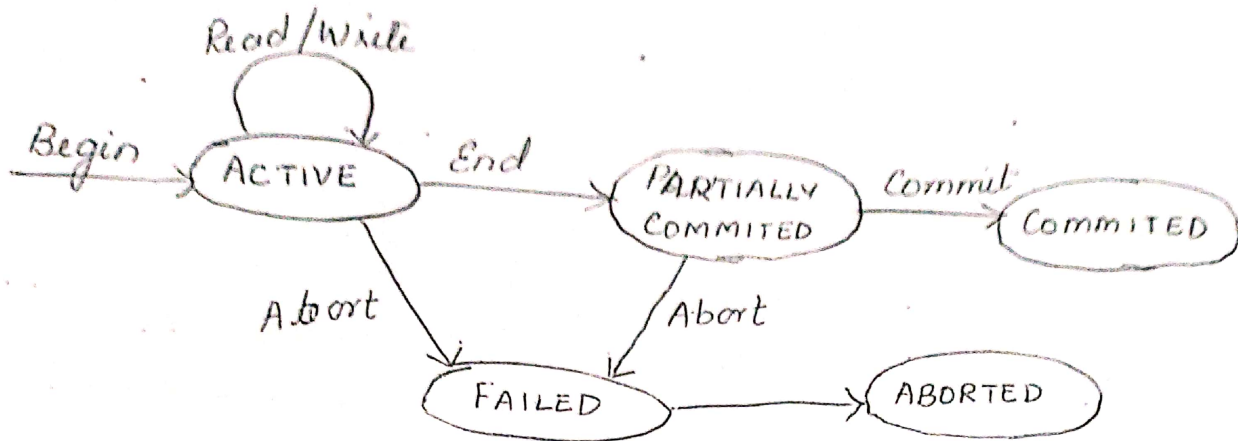
Consistency : The database should be with consistent data after the execution transaction. That is, there should not be any loss of data and this property of consistency is achieved through the atomicity property.

Isolation : A schedule has multiple may have more than one transaction in it and each transaction performs operations of various data items. Multiple transactions can access same data item. The property of isolation indicates that each transaction should work independently and should not get affected by the other transactions (until committed).

Durability : The database should store that the data and should be made available on retrieval. Durability ensures maintaining updates of committed transactions. These updates must never be lost. It is the ability of the system to recover committed transaction updates if either the S/M or the storage media fails.

(3)

State Diagram of a transaction



A transaction goes through various stages/states before it gets terminated. These states are written in the state diagram also called state transition diagram.

1. **Begin** → This operation specifies the beginning of the transaction.
2. **Read** → Performs a read on the data item in the database.
3. **Write** → Performs a write operation on the data item in the database.
4. **End** → Specifies the end of the transaction. After this operation the transaction may have the chances of failing & it enters the partially committed or ~~or~~ otherwise aborted state.
5. **Commit** → If no failure occurs, it enters the partially committed state (data in the buffer).

(4)

5. Commit $\rightarrow$ Once the transaction is committed, the data is stored in the physical storage.

6. Abort $\rightarrow$ This operation signifies that it is an unsuccessful transaction. For each aborted transaction, the changes made must be undone or rolled back to ensure atomicity.

2 operations in a db transaction $\begin{cases} \text{read} \\ \text{write} \end{cases}$

Steps involved in $R(x)$ - Read(x)

- (i) Find the address of the disk block that contains 'x'.
- (ii) Copy the disk block into the buffer in main memory.
- (iii) copy 'x' from buffer to program variable 'x'.

Steps involved in $W(x)$ - Write(x)

- (i) Find the address of the disk block that contains 'x'.
- (ii) copy the disk block into the buffer in main memory.
- (iii) Copy the p.value in the program variable 'x' into the buffer.

(9)

Inconsistency in database

When concurrent execution of transactions occur, the DBMS ensures proper consistency of the database.

The inconsistency may appear when the transactions are executed concurrently.

There are mainly 3 problems that arise due to interleaving

1. Lost Update problem (RW conflict)
2. Dirty Read Problem (WR conflict)
3. Blind write Problem (WW conflict)

1. Lost Update Problem

is a typical problem in transaction processing when 2 transactions access and update the same data from the database

Ex	S1: T1	T2
	(80) R(X)	
	(75) $X = X - N$	
	(75) W(X)	
	commit	
		R(X) (75)
		$X = X + M$ (79)
		W(X) (79)
		commit

$X = 80$
 $N = 5$
 $M = 4$

Serial Schedule

↓ Results in

$X = 79$

S2: T1	T2
R(X)	
(75) $X = X - N$	
	R(X) (80)
	$X = X + M$ (84)
	W(X) (84)
	commit
(84) W(X)	
commit	

Interleaved Schedule

↓ Results is

$X = 84$

(Lost the update made by T_1 i.e., $X = X - N$ (75).)

Dirty Read Problem (6)

Occurs when one transaction is changing the record and the other transaction can read this record before the first transaction has been committed or rolled back. This is known as dirty read scenario because there is always the possibility that the first transaction may rollback the change, resulting in the second transaction having read an invalid data.

Ex

$X = 100$ $N = 20$ $M = 10$ $Y = 50$

S1:	T1	T2
(100)	R(X)	
(80)	$X = X - N$	
(80)	W(X)	
(50)	R(Y)	
(70)	$Y = Y + N$	
(70)	W(Y)	
		R(X) (80)
		$X = X + M$ (90)
		W(X) (90)

Serial Schedule

↓ Results in

$X = 90$

$Y = 70$

S2:	T1	T2
(100)	R(X)	
(80)	$X = X - N$	
(80)	W(X)	
		R(X) (80)
		$X = X + M$ (90)
		W(X) (90)

R(Y)
 $Y = Y + N$
W(Y)

shld not execute because T1 has failed/aborted

let us say at this point T1 fails (not committed) In that case X should have 110 after T2

Inference after S2.

ie, T2 has made a dirty read/read a dirty value (wrong value) of X

But result is

$X = 90$

Y not executed

blind write

(7)

Occurs when a transaction writes a value without reading it. In particular,

Example : $X = 2$

S1:

T1	T2	T3
(2) R(X)		
(4) $X = X * 2$		
(5) $X = X + 4$		
(40) $X = X * 5$		
(40) W(X)		
(40) commit		
	W(X) (40) commit	W(X) (40) commit

Serial Schedule

Results
in
 X

$X = 40$

S2:

T1	T2	T3
(2) R(X)		
(4) $X = X * 2$		
(8) $X = X * 4$		
	W(X) (2) commit	
(10) $X = X * 5$		
(10) W(X)		
commit		
		W(X) (10) commit

$X = 2$
because
previous
actions were
not write

Interleaved Schedule

Results in
 $X = 10$

Concurrent Execution of transaction

Concurrent Execution of transactions

(i) Increases system throughput

(ii) No processes or components remain idle

(iii)

Schedule : A time ordered sequence of actions taken by one or more transactions. ⁽³⁾

Serial Schedule : A schedule that executes one transaction completely and then begins the next transaction. i.e., no interleaving.

Serializable Schedule : A schedule S_1 (Serial/Concurrent) whose effect on the db is same as the effect of its serial schedule S_2 , then the schedule S_1 is serializable schedule.

Ex

T1	T2
$A = 20$ $20 \leftarrow R(A)$ $40 \leftarrow A = A + 20$ $40 \leftarrow W(A)$ Commit	$R(A) \rightarrow 40$ $A = A + 10 \rightarrow 50$ $W(A) \rightarrow 50$ Commit

S_1 : Serial Schedule
 ↓ Results in $A = 50$

T1	T2
$20 \leftarrow R(A)$ $50 \leftarrow A = A + 20$ $50 \leftarrow W(A)$ Commit	$R(A) \rightarrow 20$ $A = A + 10 \rightarrow 30$ $W(A) \rightarrow 30$ Commit

S_2 : Interleaved Schedule
 ↓ Results is
 $A = 50$.
 (Same as S_1)

⇒ S_2 is a
 Serializable
 Schedule.

Conflict

(9)

Conflict occurs when

- (i) different transactions access same data item
- (ii) One operation should be a write operation.

3 kinds of conflicts

RW (Read-Write)
WR (Write-Read)
WW (Write-Write)

Conflict Equivalent

Two schedules are said to be conflict equivalent if they have same set of conflicts.

Conflict Serializable

If a schedule S_1 is conflict equivalent to its serial schedule S_2 . Then S_1 is conflict serializable.

Ex 1

T1	T2
$W_1(X)$	
	$R_2(X)$
$W_1(Y)$	
	$R_2(Y)$
$W_1(Z)$	$W_2(Y)$

↑ Schedule S_1

T1	T2
$W_1(X)$	
$W_1(Y)$	
$W_1(Z)$	
	$R_2(X)$
	$R_2(Y)$

Conflicts (C_1)

$W_1(X) R_2(X)$
 $W_1(Y) R_2(Y)$
 $W_1(Y) W_2(Y)$

Conflicts (C_2)

$W_1(X) R_2(Y)$
 $W_1(Y) R_2(Y)$
 $W_1(Y) W_2(Y)$

Here $C_1 = C_2$

⇒ S_1 is
conflict equivalent
& also conflict
serializable
with S_2 .

Ex: 2

(10)

T1	T2
$R_1(x)$	$W_2(x)$
$W_1(x)$	

S1

Conflicts of $S_1 = C_1$
 $R_1(x)W_2(x)$
 $W_2(x)W_1(x)$

T1	T2
$R_1(x)$	
$W_1(x)$	$W_2(x)$

S2

Conflicts of $S_2 = C_2$
 $R_1(x)W_2(x)$
 $W_1(x)W_2(x)$

$C_1 \neq C_2$.
 $\Rightarrow S_1$ is not conflict equivalent to S_2
 $\Rightarrow S_1$ is also not conflict serializable to S_2

Note : $W_1(x)W_2(x) \neq W_2(x)W_1(x)$

Testing Conflict Serializability using precedence graph

It is not always possible to test whether a given schedule is conflict serializable by manual methods because there may be thousands of transactions executing concurrently.

Therefore, a method which can test a given schedule for its conflict serializability is introduced. This is precedence graph.

Precedence graph

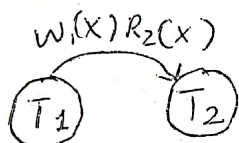
(11)

Step 1: Draw nodes corresponding to each of the transaction in schedule S.

Step 2: Draw an edge from T_i to T_j , if T_i precedes T_j and conflicts T_j (conflict actions (WW, RW, WR))

Step 3: If the directed graph has a cycle then S is not conflict serializable else S is conflict serializable.

Ex 1 given $W_1(x) R_2(x)$



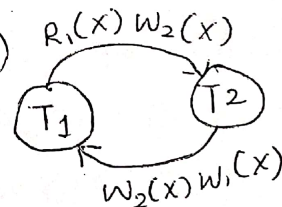
No cycle formed
So conflict serializable

Ex 2 given

T1	T2
$R_1(x)$	
$W_1(x)$	$W_2(x)$

Soln

conflicts: $R_1(x) W_2(x)$
 $W_2(x) W_1(x)$



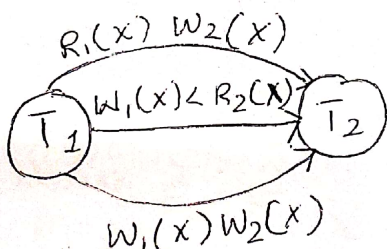
Cycle formed so not conflict serializable

Ex 3 given:

$R_1(x) < W_2(x)$, $W_1(x) < R_2(x)$, $W_1(x) < W_2(x)$,

$R_1(x) < W_2(x)$, $R_1(z) < W_2(y)$

Repeated



No cycle formed
So conflict serializable

(12)

View Equivalent Schedule

Consider two schedules S_1 and S_2 . They are said to be view equivalent if following are true

Rule (i) Initial read must be same in both S_1 & S_2

S_1	T_1	T_2
	R(A)	
		W(A)

S_2	T_1	T_2
		W(A)
	R(A)	

S_1 : T_1 reads A from db

S_2 : T_1 reads A from T_2

$\therefore S_1$ and S_2 are not satisfying first rule of view equivalence

Rule (ii) There are 2 transactions T_i & T_j , the schedules S_1 and S_2 are view equivalent if in schedule S_1 , T_i reads A that has been updated by T_j and in schedule S_2 , T_i must read A from T_j i.e., WR sequence must be same between S_1 & S_2 .

S_1	T_1	T_2	T_3
	W(A)		
		W(A)	
			R(A)

S_2	T_1	T_2	T_3
		W(A)	
	W(A)		
			R(A)

S_1 : T_3 reads A from T_2

S_2 : T_3 reads A from T_1

$\therefore S_1$ and S_2 are not satisfying second rule of view equivalence

(13)

ie (iii) Final write operation should be the same between S_1 and S_2 .

$$S_1:$$

T_1	T_2	T_3
$W(A)$		
	$R(A)$	
		$W(A)$

$$S_2:$$

T_1	T_2	T_3
	$W(A)$	
	$R(A)$	
$W(A)$		$W(A)$

S_1 : A is finally updated by T_3

S_2 : A is finally updated by T_1

$\therefore S_1$ and S_2 are not satisfying the 3rd rule of view equivalence.

View Serializable Schedule

A

ie.,

(i) If in S_1 , T_i reads the item from db first then in S_2 also T_i should do the first read from db

(ii) If in S_1 , T_i does the last write on the data item then in S_2 also T_i should do the last write on item

(iii) If in S_1 $W_j(A) < R_i(A)$ then in S_2 $W_j(A) < R_i(A)$ should exist.

View Serializable Schedule : A schedule is view serializable, if it is view equivalent to its serial schedule.

Concurrency Control

(14)

Concurrent execution of transaction is unavoidable as it gives better throughput.

Control of concurrent transaction must be taken care by the system through concurrency control schemes. There are set of protocols that will ensure serializability of schedule. The most popular and simpler one is locking technique.

Lock based concurrency control

Types $\begin{cases} \text{Binary} \\ \text{Shared \& Exclusive} \end{cases}$

Binary

(i) LOCK(x) - lock on object $x \Rightarrow L(x) = 1$.

(ii) UNLOCK(x) - unlock the object $x \Rightarrow L(x) = 0$.

Algorithm for locking an object

LOCK(x)

{

 If $L(x) = 0$

 then $L(x) = 1$

 else

 wait until $L(x) = 0$

 wake up T

}

Algorithm for unlocking ⁽¹⁵⁾ an object x

unlock(x)

{

$L(x) = 0$

If any T is waiting in the queue
then wake up T

}

Shared and Exclusive lock

In case of binary lock, even if the read-read operation occurs, there is a lock getting applied.

But R-R is not a conflict. Hence we use the concept of shared lock & exclusive lock

Algorithm for shared lock

Shared lock :

If a shared lock is requested by T , the lock manager grants it provided there is no exclusive lock on the object A i.e., $L(A) = S$.

Lock manager then increments the count of number of read locks on A by 1.

Exclusive Lock :

If an exclusive lock is requested by T , the lock manager grants it provided no other transaction holds the lock on A . If the request raised by T cannot be granted, then T is inserted into the waiting queue.

Algorithm for shared lock (16)

Algorithm S(A)

```
{
  if L(A) = 0 // no lock on A
  then
    { L(A) = S // set initial shared lock
      N(A) = 1 // S lock counter
      update lock table
    }
  else
    if L(A) = S // Already shared
    then
      { N(A) = N(A) + 1
        update lock table
      }
    else insert T into queue & suspend T. // A has an
                                              exclusive lock
}
```

Algorithm for unlocking

Algorithm U(A)

```
{
  if L(A) = 'X'
  then
    { L(A) = 0 // Release the lock
      activate a T from queue
    }
  else if L(A) = 'S'
  then
    { N(A) = N(A) - 1
      if N(A) = 0
      then
        L(A) = 0
        activate a T from queue
      }
    }
}
```


Algorithm for exclusive lock (14)

Algorithm x(A)

{ if L(A) = 0

then

{ L(A) = 'X' // set exclusive lock
update lock table

}

else // already locked
insert T into queue

}

NOTE:

- (i) A transaction obtaining a shared mode lock on a data item can only read the data item. It cannot write or update.
- (ii) A transaction obtaining exclusive mode lock can both read and write the data item.
- (iii) Several shared mode locks can be held simultaneously on a data item by different transactions.

Lock Compatibility Matrix

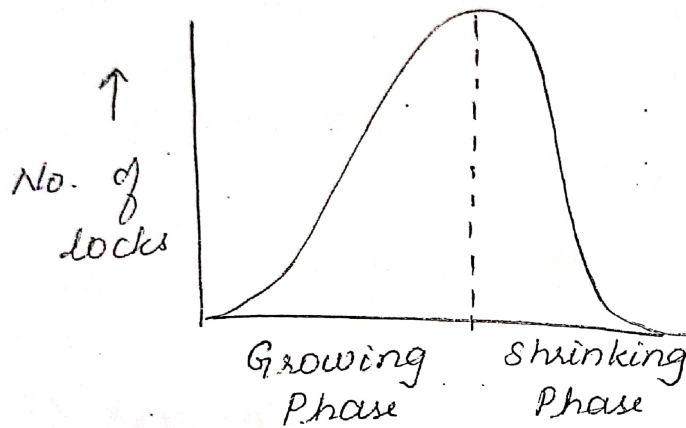
Mode of lock requested →

Mode of lock
currently
held by other
T

	U	S	X
U	-	✓	✓
S	✓	✓	
X	✓		

Two Phase Locking (18)

Growing Phase : depicts locking on data items
Shrinking Phase : depicts releasing the lock on data items.



Two Types $\left\{ \begin{array}{l} \text{Strict 2PL} \\ \text{Conservative 2PL} \end{array} \right.$

Strict 2PL

Locks data items as and when required and releases all the locks together at the end.

Conservative 2PL

Locks all the data items in the beginning and releases the locks as it finishes its intended job with the data item.

(19)

Given T as

$$R(A) \ W(A) \ R(B) \ W(B) \ R(C) \ W(C) \ R(D) \ W(D)$$
Strict 2PL

$$L(A)=1 \ R(A) \ W(A) \ L(B)=1 \ R(B) \ W(B) \ L(C)=1 \ R(C) \ W(C),$$

$$L(D)=1 \ R(D) \ W(D) \ U(A) \ U(B) \ U(C) \ U(D)$$
Conservative 2PL

$$L(A)=1 \ L(B)=1 \ L(C)=1 \ L(D)=1$$

$$R(A) \ W(A) \ U(A) \ R(B) \ W(B) \ U(B) \ R(C) \ W(C) \ U(C) \ R(D) \ W(D) \ U(D)$$
Deadlocks

T_1	T_2
$X(A)$	
$W(A)$	
	$X(B)$
	$W(B)$
	$X(A)$
	$W(A)$
$X(B)$	
$W(B)$	

Here, using Strict 2PL, a lock is performed on A since there is a $W(A)$.
 Now in T_2 we again try to write on A i.e., $W(A)$ but we have already locked A under T_1 .
 Hence there is a deadlock.

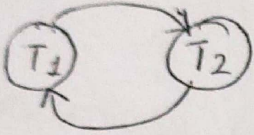
Deadlock detection using waits for graph

- Step 1: Draw one vertex for each transaction in the schedule
- Step 2: An edge $(T_i \rightarrow T_j)$ is added to the graph if and only if T_i is waiting for T_j
- Step 3: If the graph at any point of time has cycle, it implies deadlock.

(20)

Ex 1

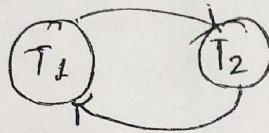
T ₁	T ₂
S(A)	
	X(A)
X(A)	



cycle formed
so deadlock

Ex 2

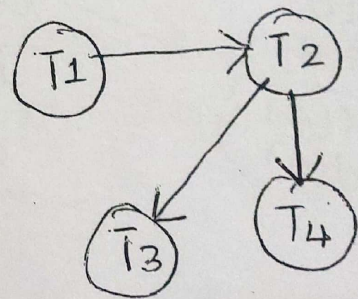
T ₁	T ₂
X(A)	
W(A)	
	X(B)
	W(B)
	X(A)
	W(A)
X(B)	
W(B)	



cycle formed so deadlock

Ex 3

T ₁	T ₂	T ₃	T ₄
		X(A)	
	X(A)		
X(A)			X(B)
	X(B)		

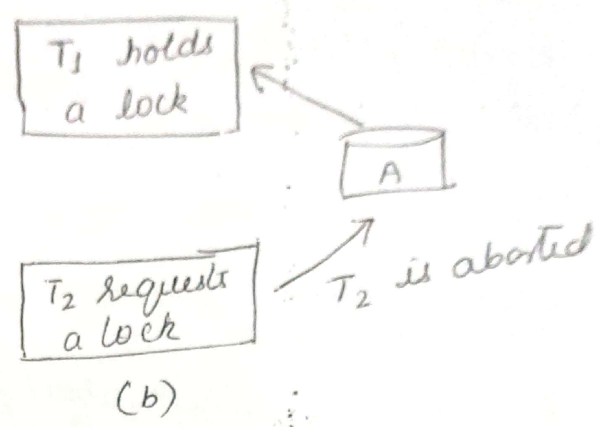
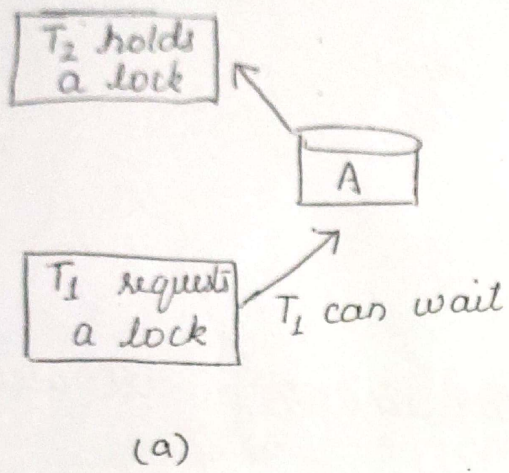


No Cycle
So no deadlock.

Deadlock Prevention

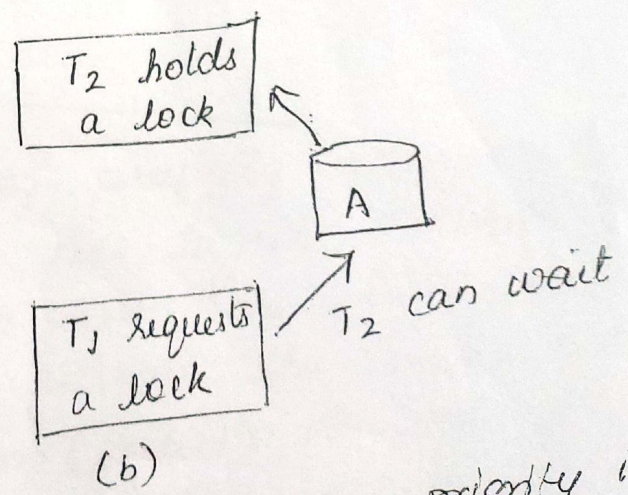
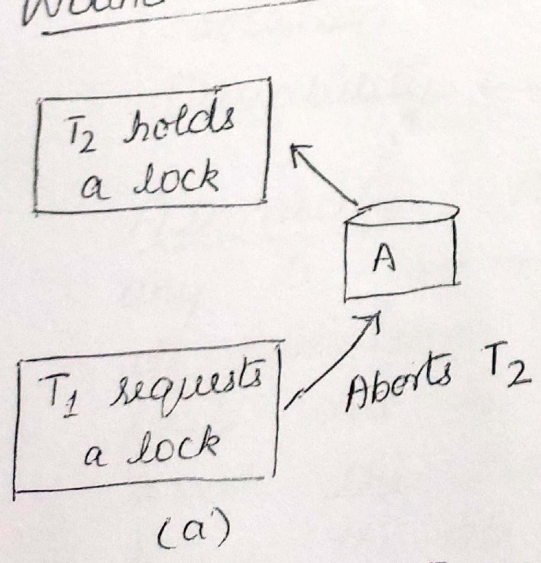
- 2 Schemes
 - Wait Die scheme
 - Wound wait scheme

I. Wait Die Scheme



T₁ → high priority transaction
T₂ → low priority transaction
(Higher priority transaction will wait if it requests whereas the lower priority transaction dies if it requests).

Wound-wait scheme



(Higher priority transaction will abort lower priority if it wants a lock on item, whereas lower priority wait if it requests for a lock on item).

I. Wait-Die Scheme (22)

- (i) If $T_i < T_j$ (T_i has higher priority than T_j), then T_i is allowed to wait otherwise T_i is aborted and rolled back.
- (ii) Lower priority transactions can never wait for higher priority transaction.
- (iii) It is a non pre-emptive technique.

II. Wound-wait Scheme

- (i) If $T_i < T_j$ then abort T_j otherwise T_i is allowed to wait.
 - (ii) Higher priority transactions can never wait lower priority transaction. Higher priority transaction forces roll back of lower priority transaction.
 - (iii) It is a pre-emptive technique.
-